# Capstone Design Document + V&V Document

Version 1
Group 13
COMPSCI 4ZP6
Dr. Mehdi Moradi
Jan 23rd, 2026


| | | |
|---|---|---|
| Abdul-Hadi Siddiqui | sidda1@mcmaster.ca | 400386070 |
| Ankush Sarkar | sarkaa16@mcmaster.ca | 400376621 |
| Asad Muhammad | muhama23@mcmaster.ca | 400368869 |
| Eduardo Salvacion | salvacie@mcmaster.ca | 400393061 |
| Jayesh Anil | anilj@mcmaster.ca | 400384263 |
| Michael Zhang | zhanz479@mcmaster.ca | 400387890 |
| Viransh Shah | shahv47@mcmaster.ca | 400394334 |

# Table of Contents

**Table of Contents** 2
**1. Purpose** 4
**2. Component Diagram** 4
**3. Requirements** 5
3.1 Presentation Layer 5
3.1.1 Priority 0 5
3.1.2 Priority 1 6
3.1.3 Priority 2 6
3.2 Business Layer 7
3.2.1 Priority 0 7
3.2.2 Priority 1 7
3.2.3 Priority 2 7
3.3 Data Layer 7
3.3.1 Priority 0 7
3.3.2 Priority 1 7
3.4 Gemini 8
3.4.1 Priority 0 8
3.5 Google ML Kit + Expo Voice 8
3.5.1 Priority 0 8
3.5.2 Priority 1 8
3.6 State Management 8
3.6.1 Priority 0 8
3.6.2 Priority 1 8
3.7 Database/Remote Infrastructure 9
3.7.1 Priority 0 9
3.7.2 Priority 1 9
**4. Components** 9
4.1 Presentation Layer 9
4.1.1 Normal Behaviour 9
4.1.2 API and Data Structures 9
4.1.3 Implementation 10
4.1.4 Potential Undesired Behaviors 10
4.2 Business Layer 10
4.2.1 Normal Behaviour 10
4.2.2 API and Data Structures 10
4.2.3 Implementation 10
4.2.4 Potential Undesired Behaviors 10
4.3 Data Layer 11
4.3.1 Normal Behaviour 11

4.3.2 API and Data Structures..... 11
4.3.3 Implementation..... 11
4.4.4 Potential Undesired Behaviors..... 11
4.4 Gemini.....12
4.4.1 Normal Behaviour.....12
4.4.2 API and Data Structures.....12
4.4.3 Implementation.....12
4.4.4 Potential Undesired Behaviors..... 12
4.5 Google ML Kit + Expo Voice..... 12
4.5.1 Normal Behaviour.....12
4.5.2 API and Data Structures.....12
4.5.3 Implementation.....12
4.5.4 Potential Undesired Behaviors.....13
4.6 State Management.....13
4.6.1 Normal Behaviour.....13
4.6.2 API and Data Structures.....13
4.6.3 Implementation.....13
4.6.4 Potential Undesired Behaviors.....13
4.7 Database/Remote Infrastructure..... 13
4.7.1 Normal Behaviour.....13
4.7.2 API and Data Structures.....14
4.7.3 Implementation..... 14
4.7.4 Potential Undesired Behaviors.....14
**5. UI Details..... 15**
5.1 General..... 15
5.2 Color.....15
5.3 Font..... 15
5.4 Placement.....15
5.5 Figma.....15
**6. V&V Plan..... 15**
6.1 Presentation Layer..... 15
6.1.1 Unit Tests.....15
6.1.2 Performance Tests and Metrics.....16
6.1.3 Snapshot Tests..... 16
6.2 Business Layer.....16
6.2.1 Unit Tests.....16
6.2.2 Performance Tests and Metrics.....16
6.3 Data Layer.....17
6.3.1 Unit Tests..... 17
6.3.2 Performance Tests and Metrics.....17
6.4 Gemini.....17

6.4.1 Unit Tests..... 17
6.4.2 Performance Tests and Metrics.....17
6.5 Google ML Kit + Expo Voice.....17
6.5.1 Unit Tests..... 17
6.5.2 Performance Tests and Metrics.....18
6.6 State Management.....18
6.7 Database/Remote Infrastructure.....18
6.7.1 Unit Tests..... 18
6.7.2 Performance Tests and Metrics.....18

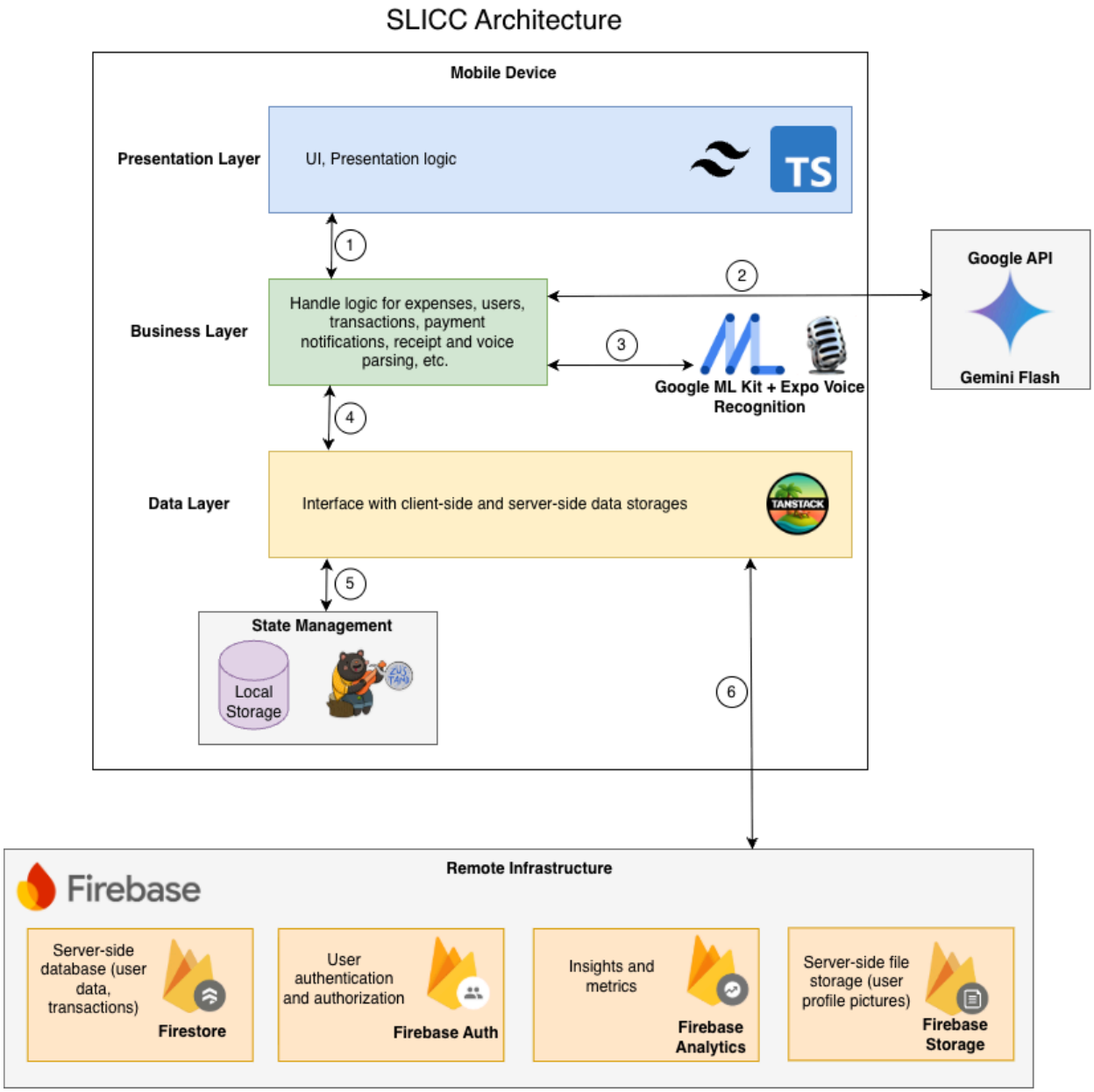
# 1. Purpose

The Slicc project simplifies group expense management by combining event planning, expense splitting through OCR and voice recognition, and payment coordination in a single application. Key features include automatic receipt scanning, voice-based bill splitting, payment tracking by expense creators, configurable tax/tip handling, group-based event organization, and flexible payment integration for both app users and non-users.

This design document outlines the technical architecture, component design, and validation/verification plan for the Slicc system. It explains the relationship between components and how the system meets the requirements defined in Software Requirements Specification (SRS).

# 2. Component Diagram

Components:
Presentation Layer
Business Layer
Data Layer (Move React Query here)
Gemini
Google ML Kit + Expo Voice
State Management (Combine Zustand + Local data)
Database/Remote Infrastructure




# 3. Requirements

## 3.1 Presentation Layer

### 3.1.1 Priority 0

- The host user should be able to manually add a list of expenses
- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses
- The host user should be able to manually split the expenses among other people or users

- The host user should be able to vocally split the expenses among other payee users through the app (voice detection)
- User authentication
    - The user should be able to sign-up and login with their email, or associated Google/Apple account
    - The user should be able to link their e-transfer email or paypal/venmo details to seamlessly make payments when needed
    - The user should be able to access their recent expenses and expense history

### 3.1.2 Priority 1

- The user can create events and add other users
    - Users will be added through their unique identifier
    - The user can also create stand-in users for people who do not have the app
        - The stand-in can have either a phone number or email for the purpose of sending a payment reminder
        - The user can also give the stand-in a location for recommendation purposes
    - Users of the app should be able to add dates and times they are available
        - Availability for stand-in users can be changed by other app users that are in the group
    - The user can see a list of other event members
    - The user can set a location for the event
- The user can search through nearby locations
    - Locations can be filtered and sorted by based on an input location
    - Locations can be filtered and sorted by based on average of event members location
    - Locations can be filtered and sorted by based on dietary restrictions for

### 3.1.3 Priority 2

- The user can create events and add other users
    - The user can export the event date and time to their Google/Apple calendar
- The user can access an in-app map that displays recommended locations of interest (restaurants, karaoke, etc.)
- Event members should be able to access a shared noteboard
    - The user can access the noteboard quickly from anywhere in the event interface

## 3.2 Business Layer

### 3.2.1 Priority 0

- The host user should be able to manually add a list of expenses
- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses
- The host user should be able to manually split the expenses among other people or users
- The host user should be able to vocally split the expenses among other payee users through the app (voice detection)
- The payee user should be redirected to their preferred payment method when a payment is requested
    - For people who need to make payments but do not have the app, the host user can input their email or phone number to send an payment notification

### 3.2.2 Priority 1

- The user can search through nearby locations
    - Locations can be filtered and sorted by based on an input location
    - Locations can be filtered and sorted by based on average of event members location
    - Locations can be filtered and sorted by based on dietary restrictions for

### 3.2.3 Priority 2

- The user can create events and add other users
    - The user can export the event date and time to their Google/Apple calendar
    - The user can send and receive messages from other event members

## 3.3 Data Layer

### 3.3.1 Priority 0

- User authentication
    - The user should be able to sign-up and login with their email, or associated Google/Apple account
    - The user should be able to access their recent expenses and expense history

### 3.3.2 Priority 1

- The user can create events and add other users
    - Users will be added through their unique identifier
    - The user can also create stand-in users for people who do not have the app

        - The stand-in can have either a phone number or email for the purpose of sending a payment reminder
        - The user can also give the stand-in a location for recommendation purposes
    - Users of the app should be able to add dates and times they are available
    - The user can see a list of other event members
    - The user can set a location for the event

## 3.4 Gemini

### 3.4.1 Priority 0

- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses

## 3.5 Google ML Kit + Expo Voice

### 3.5.1 Priority 0

- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses
- The host user should be able to vocally split the expenses among other payee users through the app (voice detection)

### 3.5.2 Priority 1

- The app can be used offline
    - All expense splitting functionality should be available and work offline
        - The user can manually input a list of expenses
        - The user can scan a receipt and get a parsed list of expenses
        - The user can manually split the list of expenses
        - The user can split the bill through vocal recognition
    - Offline expense history will be merged when the user comes back online
        - Only if some user has been logged in prior

## 3.6 State Management

### 3.6.1 Priority 0

- User authentication
    - The user should be able to access their recent expenses and expense history

### 3.6.2 Priority 1

- The app can be used offline
    - All expense splitting functionality should be available and work offline

        - The user can manually input a list of expenses
        - The user can scan a receipt and get a parsed list of expenses
        - The user can manually split the list of expenses
        - The user can split the bill through vocal recognition
    - Offline expense history will be merged when the user comes back online
        - Only if some user has been logged in prior

## 3.7 Database/Remote Infrastructure

### 3.7.1 Priority 0

- User authentication
    - The user should be able to sign-up and login with their email, or associated Google/Apple account
    - The user should be able to access their recent expenses and expense history

### 3.7.2 Priority 1

- The user can create events and add other users
    - Users will be added through their unique identifier
    - The user can also create stand-in users for people who do not have the app
        - The stand-in can have either a phone number or email for the purpose of sending a payment reminder
        - The user can also give the stand-in a location for recommendation purposes
    - Users of the app should be able to add dates and times they are available
        - Availability for stand-in users can be changed by other app users that are in the group
    - The user can see a list of other event members
    - The user can set a location for the event

# 4. Components

## 4.1 Presentation Layer

### 4.1.1 Normal Behaviour

- This Layer is responsible for all the front-end that the user sees and interacts with on the app.

### 4.1.2 API and Data Structures

- User interactions such as button presses, voice input, camera input to operate the app. Data is structured as React components, eventually mapping to either Android or iOS native UI.

### 4.1.3 Implementation

- Stack: React Native/Expo (SDK 52), TypeScript, NativeWind, Expo Router, React Hook Form + Zod.
- Files: Root layout (src/app/_layout.tsx), dashboard (src/app/(app)/index.tsx), expense screens (src/app/expense/*), camera (src/app/reciept-camera/index.tsx), components (src/components/*).
- Routing: Root Layout with providers -> Authenticated Routes (tabs: home, groups, notifications, settings) -> Expense/Camera/Auth screens. Zustand integration enables automatic re-rendering

### 4.1.4 Potential Undesired Behaviors

- Cross-Platform Inconsistencies: Mitigation: Platform-specific testing and styling.
- Keyboard Overlap: Mitigation: KeyboardAvoidingView, react-native-keyboard-controller.
- State Loss: Mitigation: Expo Router persistence, draft storage.

## 4.2 Business Layer

### 4.2.1 Normal Behaviour

- This layer is responsible for handling logic for expenses, users and transactions for each user. This also handles payment notifications, receipt scanning and voice parsing for users to add the transactions on their receipt and from user input.

### 4.2.2 API and Data Structures

- Calculate item amount split between users during the creation of expense.
- Parsing receipt scanner API output.
- Parsing voice recognition API output.
- Data structure used will be JSON

### 4.2.3 Implementation

- Stack: TypeScript, Axios, Zod validation, custom error classes.
- Files: src/api/expenses/, src/api/items/, src/api/people/, src/api/camera-receipt/extract-receipt-info.tsx, src/lib/utils.ts.
- Receipt Flow: Preprocess -> Gemini API -> validate -> confidence scores -> flag low confidence.
- Splitting: Subtotal -> items per person -> distribute tax -> distribute tip -> totals
- Voice: Speech-to-text -> parse intent -> extract entities -> fuzzy match -> execute -> confirm.

### 4.2.4 Potential Undesired Behaviors

Workflow: Input (voice/receipt/manual) -> Validation and split calculation -> User review/edit -> Atomic save (or offline queue + retry)

- Voice errors: Mitigated through manual entry and editable extracted items before confirmation.
- Calculation errors: Mitigated through tax-aware rounding, per-person recalculation, and numeric input validation.
- Transaction and offline failures: Mitigated through atomic batch writes and queue-based reconnect/interval retry.
- Receipt omissions: Partial mitigation through user review/edit; no automatic completeness check.

## 4.3 Data Layer

### 4.3.1 Normal Behaviour

- The Data Layer manages data retrieval, caching, persistence, and synchronization between the client and Cloud Firestore.

### 4.3.2 API and Data Structures

- The Data Layer exposes queries and mutations to the firestore and validates the JSON results with predefined ZOD schemas
    - Expense
        - create expense, delete expense, update user payment status, get expense by id, get event expenses, add user to expense
    - User
        - get user by id, get user by username, create user, set display name, update display name
    - Item
        - create item in expense, update item details, delete items, assign item to user
    - Groups
        - create group, update group details, add member to group, remove member from group
    - Events
        - create event, update event details,  send event invite to user, add expense to event

### 4.3.3 Implementation

- Stack: TanStack Query v5, Firebase Firestore, MMKV, custom sync engine.
- Files: src/api/common/api-provider.tsx, src/api/common/client.tsx, src/api/expenses/, src/api/items/, src/api/users/, src/api/groups/ src/api/events/.
- Offline Sync: Queue tracks operations -> addToQueue persists -> processQueue executes sequentially -> retry failed operations.

### 4.4.3 Potential Undesired Behaviors

- Sync Conflicts: Mitigation: Last-write-wins.
- Stale Data: Mitigation: StaleTime tuning, pull-to-refresh, real-time listeners.
- Memory Leaks: Mitigation: Cache limits, garbage collection.

## 4.4 Gemini

### 4.4.1 Normal Behaviour

- Google Gemini is used for text recognition from receipts scanned by the users. Only the items and their prices are recorded from the scanned receipts. The items and price for each item is coupled together and itemized based on what the API outputs after scanning the receipt.

### 4.4.2 API and Data Structures

- The Gemini API accepts a preprocessed, base64-encoded receipt image as input and returns structured JSON containing extracted item names, prices, and confidence scores.

### 4.4.3 Implementation

- Stack: Gemini 2.5 Flash, expo-camera, expo-image-manipulator.
- File: src/api/camera-receipt/extract-receipt-info.tsx.
- Flow: Resize to 1024px -> compress 80% -> JPEG -> base64 -> POST to API -> parse JSON -> validate -> return with confidence scores.

### 4.4.4 Potential Undesired Behaviors

Workflow: Gemini attempt -> Timeout/progress handling -> ML Kit fallback -> User correction -> Save

- Latency: Mitigated through loading state and timeout guards.
- Service dependency/cost: Mitigated through reduced image quality and on-device fallback.
- Partial extraction: Partial mitigation through user review/edit before save.
- Item-count verification: No mitigation

## 4.5 Google ML Kit + Expo Voice

### 4.5.1 Normal Behaviour

- Expo is used for the voice to text pipeline where user speech is transcribed and itemized based on what the user says on the microphone. The items and price mentioned by the user are coupled together and itemized.

### 4.5.2 API and Data Structures

- Expo voice recognition returns an object with the voice and the metadata which includes the transcript of microphone input as an array of Javascript objects.
- Input is parsed using a parts of speech tagger.

### 4.5.3 Implementation

- Stack: Firebase ML Kit, expo-speech/react-native-voice.
- Files: ML Kit in src/api/camera-receipt/extract-receipt-info.tsx, voice in Business Layer.
- Voice Flow: Set listeners -> start recognition (en-US) -> return promise with transcript and confidence -> error handling

### 4.5.4 Potential Undesired Behaviors

Workflow: On-device recognition -> Parsed draft -> Manual correction -> Final confirmation

- Offline/noisy recognition: Mitigated through on-device paths and editable fields.
- Missing details: Partial mitigation through manual override before save.
- Count/mismatch warnings: No mitigation.

## 4.6 State Management

### 4.6.1 Normal Behaviour

- The State Management component is responsible for maintaining client-side application state that must persist across screens and sessions. This includes authentication state, user preferences etc.

### 4.6.2 API and Data Structures

- User settings, work in progress expenses, and user authentication are stored via JSON objects for client-side persistence.

### 4.6.3 Implementation

- Stack: Zustand, react-native-mmkv.
- Files: src/lib/store.ts, src/lib/storage.tsx, src/lib/auth/index.tsx.
- Offline Queue: NetInfo monitors connectivity -> auto-update offline status -> process queue on restore -> sequential execution -> retry failed operations.

### 4.6.4 Potential Undesired Behaviors

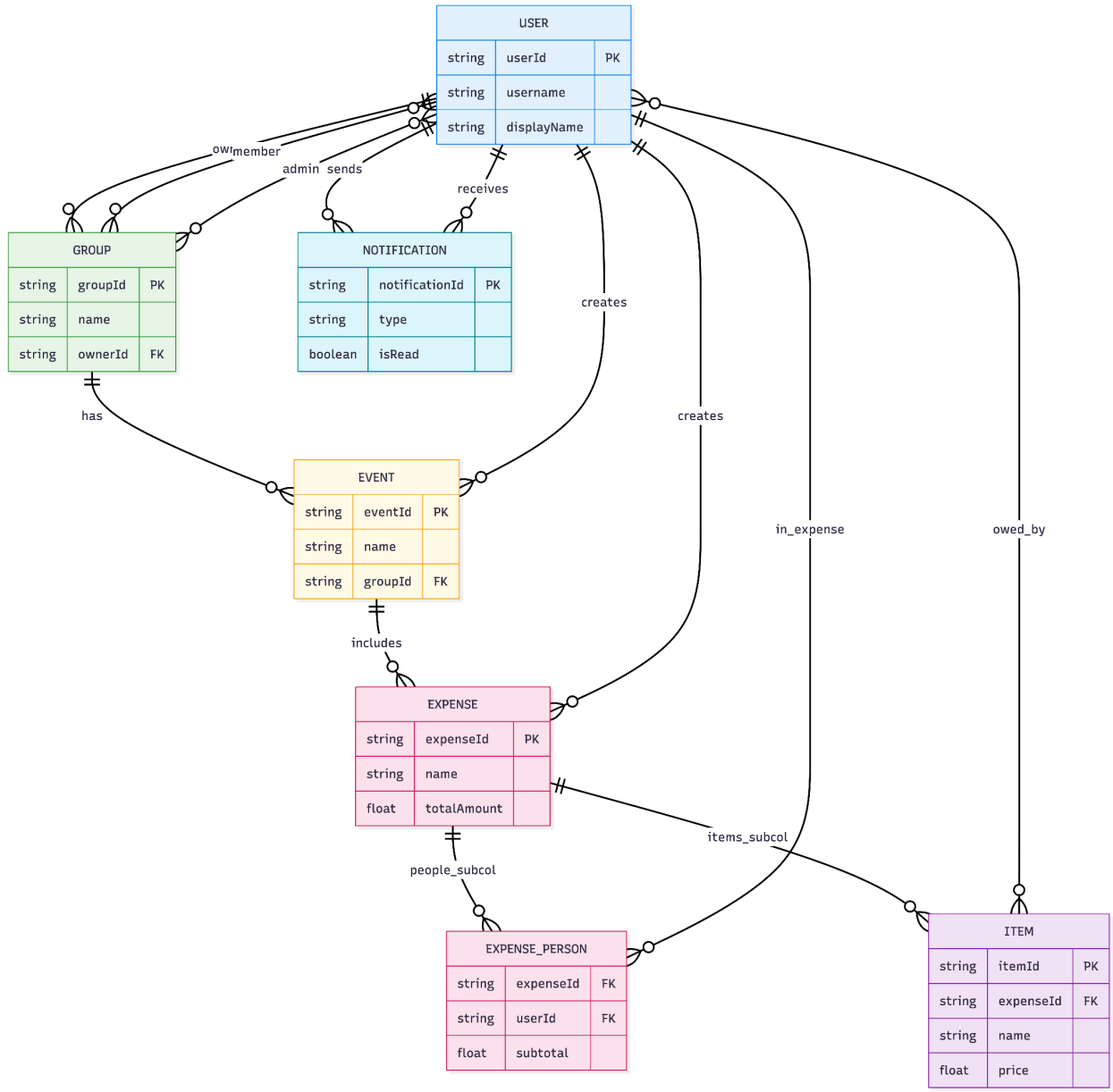
- State Bloat: Mitigation: Keep minimal global state, use React Query for server state, implement slicing.
- Race Conditions: Mitigation: Atomic updates, optimistic locking, avoid derived state.

# 4.7 Database/Remote Infrastructure

## 4.7.1 Normal Behaviour

- The Database and Remote Infrastructure component is responsible for securely storing and retrieving persistent application data and handling user authentication.

## 4.7.2 API and Data Structures



### 4.7.3 Implementation

- Stack: Cloud Firestore, Firebase Auth, Firebase Storage, Firebase Analytics, @react-native-firebase SDK.
- Config: google-services.json (Android), GoogleService-Info.plist (iOS), firestore.rules, storage.rules.

### 4.7.4 Potential Undesired Behaviors

Workflow: Auth -> Write attempt -> Error handling/offline path -> Retry/sync -> Persisted state

- Authentication and sync/network issues: Mitigated through user-facing auth errors, offline queue retry, and reachability handling.
- Data integrity: Partial mitigation through atomic expense writes and schema-validated receipt parsing.
- Permission-rule mistakes: No additional client-side mitigation beyond assumed rules.
- Duplicate transactions/idempotency: No mitigation.
- Storage failures: Partial mitigation only; no unified retry/backoff strategy.

# 5. UI Details

## 5.1 General

The general flow of our application is inspired by many existing apps such as Wealthsimple and Whatsapp. We plan on having 5 tabs at the bottom of the screen to switch between the “main screens”, which are: Home/Dashboard, Expenses, Social, Explore, Settings. Our plan is to have the app feel clean and without too much clutter.

## 5.2 Color

We are mainly using a black and white color palette. We use greys as the background for dark mode and white at the background for light mode. We also use a blue for an accent for small details and buttons, but this may be adjusted during development. We plan on sticking to a fairly limited color palette to keep our app minimal and sleek.

## 5.3 Font

The app uses the Inter font for almost all the text. This choice was made because it is sans serif, to maintain the minimal and clean style of the app. We use a

standardized font size for normal text and headers. We also decided to only use different fonts for emphasis or stylization of non-functional text.

## 5.4 Placement

The placement of items stays semi-consistent in the app. We try to maintain a bottom section that holds progressions buttons and showcases “summary” info. The top contains a header and buttons that allow the user to go back if needed.

## 5.5 Figma

The current/rough designs can be found in the Figma below.
Link: [Figma]

# 6. V&V Plan

## 6.1 Presentation Layer

### 6.1.1 Unit Tests

Presentation-layer unit testing verifies component rendering, UI behavior, and user interactions for reusable interface elements and key form surfaces. Current automated coverage includes component tests for title and button presentation/ interaction behavior, shared input controls (input, checkbox/radio/switch, select), and login-form validation interactions. React Testing Library is used to render components and simulate user actions.

Multi-screen end-to-end flows (for example full create-group/ create-event/ create-expense navigation sequences) remain validated through manual or E2E scripts rather than isolated Jest unit tests.

### 6.1.2 Performance Tests and Metrics

The measured metric is initial render/interactivity timing on representative UI components. Automated checks assert that initial render remains below 2 seconds for covered components. Uses component-level timing checks as a deterministic proxy; full-screen and low-end device validation is treated as manual/E2E performance verification.

### 6.1.3 Snapshot Tests

Snapshot testing is implemented for presentation components using Jest snapshots (React Native tree snapshots), currently including Title and Button. These snapshots lock expected rendered structure and styling contracts for regression

detection. Pixel/screenshot similarity thresholds (e.g., ~90%) are enforced in visual/E2E pipelines and are not duplicated by unit-level Jest snapshots.

## 6.2 Business Layer

### 6.2.1 Unit Tests

Business-layer unit tests verify correctness of expense and domain logic across utility and hook-level behavior. Current coverage includes:

- balance aggregation and payer/participant owed calculations
- split/share logic and shared helper behavior
- event categorization and relevance selection
- payment preference/deep-link fallback behavior
- default rate resolution and option normalization (tip/dietary)
- settings helper validation/sanitization logic
- group preference state helpers (pin/unread persistence)
- mutation-side cache invalidation behavior after create operations
- product-level KPI checks via src/vv/vv-product-metrics.test.ts for workflow completion-time budget, recommendation quality (Precision, Recall, F1), and reliability proxy checks (synthetic voice failure rate)

These tests use deterministic mocks and edge-case inputs to verify correctness and guard against regression in application rules.

### 6.2.2 Performance Tests and Metrics

Business-logic responsiveness is tested via repeated expense-share computation and measured CPU-time per operation. Acceptance remains aligned to a <200 ms responsiveness budget, with automated tests enforcing a non-flaky per-operation bound under repeated iterations.

## 6.3 Data Layer

### 6.3.1 Unit Tests

Data-layer unit tests validate fetch, cache, persistence, and mutation behavior with mocked dependencies. Coverage includes:

- query composition and aggregation via balance hooks
- successful mutation behavior with dependent query invalidation/refetch expectations
- client-side settings persistence hook behavior

Mocked responses are used for deterministic success/failure scenarios and repeatable data-layer verification.

### 6.3.2 Performance Tests and Metrics

The data-layer performance metric is mocked API responsiveness under normal expected conditions. Automated checks validate balance aggregation completion within a <500 ms budget when each mocked fetch is fast, supporting responsive UI data availability expectations.

## 6.4 Gemini

### 6.4.1 Unit Tests

Gemini-related tests validate multimodal request formatting and receipt response handling using mocked Gemini client behavior. Coverage verifies:

- expected request payload structure (model + image inlineData + prompt shape)
- successful model-text handling
- failure-path propagation for client/API errors
- parser success/failure accounting through sampled CORD receipt evaluation scripts

### 6.4.2 Performance Tests and Metrics

Automated tests validate receipt extraction under a < 5 s budget using mocked flows (no live Gemini call in this harness). In addition to latency, quality-oriented checks are enforced so verification is not runtime-only. Current OCR metrics include:

- total amount extraction accuracy (critical target: >= 90%)
- line-item extraction quality (precision, recall, and F1; acceptable target: >= 80% quality level)
- price error (mean relative error)
- OCR failure rate (<= 10%)
- aggregate latency reporting (avg and p95) under the 5-second budget
- Metrics are reported during test execution by the CORD sampled script for traceable V&V reporting

## 6.5 Google ML Kit + Expo Voice

### 6.5.1 Unit Tests

Voice/NLP coverage validates transcript-to-structured-slot extraction behavior for item name and amount parsing. Tests include normal inputs and unclear/ partial/ noisy transcript cases, including edge conditions where extractable noun/price data is missing. In addition, intent-level correctness is validated via deterministic labeled transcript cases reporting intent-detection precision and recall.

### 6.5.2 Performance Tests and Metrics

Voice parsing tests enforce a < 3 s latency budget and include correctness-oriented metrics to evaluate extraction quality in addition to runtime. Current voice metrics include:

- word error rate (WER) target <= 15%
- relative price error for extracted amount fields
- intent-detection precision target >= 85%
- intent-detection recall target >= 80%
- reliability proxy checks for failure-rate budget (<= 10%) with documented manual-input fallback expectation for remaining failures

All checks are executed with deterministic transcript-based simulation (no live microphone/audio pipeline in unit Jest).

## 6.6 State Management

State management remains out of standalone test scope as a separate layer. Its behavior is validated indirectly through presentation, business, and data-layer tests (including hooks and helper/state utility coverage).

## 6.7 Database/Remote Infrastructure

### 6.7.1 Unit Tests

Automated test coverage for infrastructure-adjacent logic currently focuses on client-side schema validation and cryptographic primitives relevant to secure data handling (e.g., key generation, wrap/unwrap, encrypt/decrypt, ratcheting behavior).

Direct Firebase auth-rule/storage enforcement testing is treated as Emulator/ integration scope rather than Jest unit scope.

### 6.7.2 Performance Tests and Metrics

Infrastructure latency targets remain:

- database operations average <500 ms,
- authentication responses <1 s.

These metrics are validated through Emulator or staging instrumentation/integration measurements.

Based on a 10-sample deterministic evaluation run, the system exceeded all defined OCR and voice-processing performance and quality targets. OCR extraction achieved a 100% parse success rate with zero failures, perfect line-item precision/recall (F1 = 1.0), and a mean total-amount accuracy of 99.5%, significantly surpassing the ≥90%

requirement. Latency performance was well within budget, with 1 ms average and 4 ms p95, far below the 5-second threshold. Voice processing similarly met all targets, achieving a mean word error rate (WER) of 11.1% (≤15% goal), and perfect intent detection (precision = 1.0, recall = 1.0), with a synthetic failure rate of 10% meeting the defined upper bound. Slot extraction tests demonstrated negligible price error and sub-millisecond to low-millisecond latency, remaining well under the 3-second constraint. Overall, the results confirm that both OCR and voice pipelines satisfy and, in most cases, substantially exceed the system's V&V performance and accuracy requirements under controlled test conditions.

## OCR (CORD-v1 style harness, mocked JSON)

| Metric | Value | Goal / budget | Pass |
|---|---|---|---|
| Samples | 10 | — | — |
| Parse success rate | 1.000 | — | — |
| OCR failure rate | 0.000 | ≤ 0.1 | yes |
| Line precision | 1.000 | — | — |
| Line recall | 1.000 | — | — |
| Line F1 | 1.000 | ≥ 0.8 | yes |
| Mean total-amount accuracy | 0.995 | ≥ 0.9 | yes |
| Mean price APE | 0.0029 | ≤ 0.1 | yes |
| Avg parse latency | 1.00 ms | < 5000 ms | yes |
| p95 parse latency | 4.00 ms | < 5000 ms | yes |

## Speech-to-text (synthetic / harness)

**Product-level (intent + WER targets from V&V plan)**

| Metric | Value | Goal | Pass |
|---|---|---|---|
| Mean WER (reference pairs) | 0.111 | ≤ 0.15 | yes |
| Intent precision | 1.000 | ≥ 0.85 | yes |
| Intent recall | 1.000 | ≥ 0.8 | yes |
| Synthetic voice failure rate | 0.10 | ≤ 0.1 | yes |

**Slot extraction (POS tagger + phrase match; latency bound Section 6.5.2)**

| Transcript | Latency (ms) | WER vs gold name | Rel. price error |
|---|---|---|---|
| burger $9.99 | 120.00 | 0.000 | 0.0000 |
| large fries cost $3.50 | 0.00 | 0.500 | 0.0000 |

Latency goal per case: < 3000 ms (Jest harness).